



Unidad 5a : Visualización en web I

Introducción a Streamlit

Widgets de Streamlit

Gráficas interactivas

Visualización de mapas en Streamlit

Ejemplo paso a paso

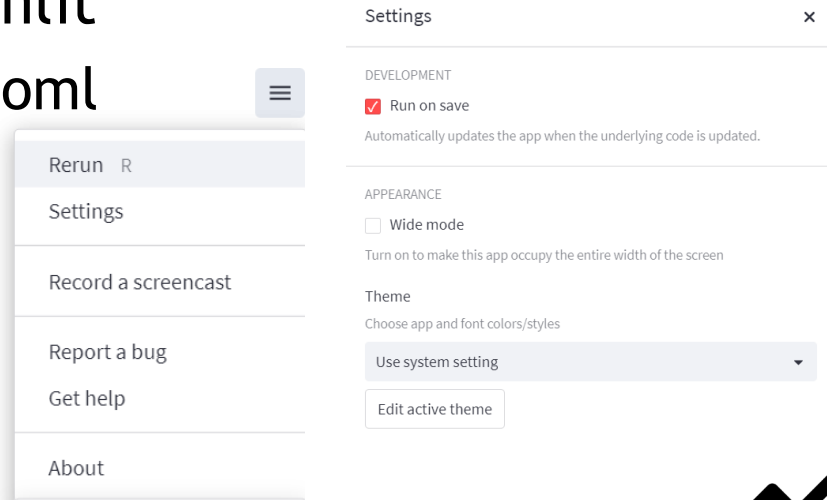
- ▶ Streamlit es una librería de código abierto que permite crear aplicaciones web para visualización de datos
- ▶ Se creó en 2019 y ha tenido un gran crecimiento
- ▶ Es muy sencilla, en pocos minutos puedes crear una aplicación interesante
- ▶ Se pueden añadir visualizaciones de todas las librerías estudiadas



- ▶ Instalación:
 - ▶ `pip install streamlit`
- ▶ Para probar que la instalación funciona:
 - ▶ `streamlit hello`
- ▶ Para ejecutar un script:
 - ▶ `streamlit run myfile.py`
 - ▶ El código puede estar en la web (GitHub)
- ▶ En máquina virtual especificar un puerto:
 - ▶ `streamlit run -server.port ???? myfile.py`
- ▶ Importación: `import streamlit as st`



- ▶ Podemos modificar la app mientras se ejecuta y ver inmediatamente los cambios:
 - ▶ Activar la opción “Run on save” en Settings
 - ▶ O crear una carpeta .streamlit
 - ▶ Añadir un fichero: config.toml
 - ▶ Incluir:
[server]
runOnSave = true



- ▶ Cada vez que se interactúa con la interfaz el script se ejecuta desde el principio
- ▶ Si hay partes de código muy costosas de ejecutar **se pueden almacenar en caché** poniendo `@st.cache_data` o `@st.cache_resource` delante de la función
- ▶ Permite múltiples páginas



- ▶ `st.write` es la orden básica para visualizar datos: texto, dataframes, etc.
- ▶ `st.write_stream`: escribe un flujo de datos utilizando el efecto máquina de escribir
- ▶ Si escribes el nombre de una variable o cadena de texto también la visualiza con `st.write`
- ▶ Hay órdenes específicas para tipos de datos, como, por ejemplo: `st.dataframe` o `st.table`



- ▶ También se pueden dibujar sencillas gráficas de líneas o de barras:

```
st.line_chart(df.Casos)
```

```
st.bar_chart(df.UCI)
```

- ▶ O dibujar puntos en un mapa con una DF con localizaciones geográficas:

```
st.map(map_data)
```



- Estos son algunos de los widgets disponibles:

`st.slider()`



`st.text_input()`

5 al cuadrado es 25

Dime tu nombre:

Xavi

`st.selectbox()`

¿Cuál es tu número favorito?

1

1

2

3

4

Tu nombre es: Xavi

`st.checkbox()`

☒ DataFrame Visible

	Mes	Casos	Hosp	UCI
0	enero	177643	12475	996
1	febrero	33727	3278	333
2	marzo	4457	482	75
3	abril	4613	442	69
4	mayo	3835	259	33



- ▶ El diseño de la página se organiza por columnas
- ▶ Se puede utilizar un panel a la izquierda:
`st.sidebar.(widget)`
- ▶ Se pueden añadir columnas:
`izq, der = st.columns(2)`
- ▶ Para indicar que elementos se colocan en una columna:
`with izq:`
`elementos`



- ▶ También se pueden añadir barras de progreso:
 - ▶ `bar = st.progress()`
 - ▶ `bar.progress()`
- ▶ En Ejemplo1.py se pueden ver todas las funciones explicadas

<https://docs.streamlit.io/library/get-started/main-concepts>



- ▶ Aplicaciones multipágina:
 - ▶ A medida que las aplicaciones crecen, resulta útil organizarlas en varias páginas
 - ▶ Esto facilita la gestión de la aplicación para el desarrollador y la navegación para el usuario
- ▶ Se realiza de esta manera:
 - ▶ Crear un script de punto de entrada para definir y conectar las páginas
 - ▶ Crear scripts de Python separados para cada página
 - ▶ Usar `st.Page` para definir las páginas y `st.navigation` para conectarlas
 - ▶ Por cada página se indica:
 - ▶ Ruta y script, Título e icono (<https://fonts.google.com/icons>)



► Script principal:

```
page = st.navigation( {"Pages": [  
    st.Page(dir_path / "hello.py", title="Hello",  
        icon=":material/waving_hand:"),  
    st.Page(dir_path / "dataframe_demo.py", title="DataFrame demo",  
        icon=":material/table:"),  
    st.Page(dir_path / "plotting_demo.py", title="Plotting demo",  
        icon=":material/show_chart:"),  
    st.Page(dir_path / "mapping_demo.py", title="Mapping demo",  
        icon=":material/public:"),  
    st.Page(dir_path / "animation_demo.py", title="Animation demo",  
        icon=":material/animation:")]})  
page.run()
```



▶ hello.py:

```
st.set_page_config(page_title="Hello",  
page_icon=":material/waving_hand:")  
st.title("Welcome to Streamlit! 🙌") ...
```

▶ dataframe_demo.py:

```
st.set_page_config(page_title="DataFrame demo",  
page_icon=":material/table:")  
st.title("DataFrame demo") ...
```

▶ plotting_demo.py:

```
st.set_page_config(page_title="Plotting demo",  
page_icon=":material/show_chart:")  
st.title("Plotting demo") ...
```

...



- ▶ Aquí tienes toda la documentación:

<https://docs.streamlit.io/library/get-started/multipage-apps/create-a-multipage-app>

- ▶ Aquí tienes un ejemplo completo:

<https://github.com/streamlit/streamlit/tree/develop/lib/streamlit/hello>

-> Multipagina.zip



- ▶ En resumen, el modelo de ejecución es:
 - ▶ Las aplicaciones Streamlit son scripts de Python que se ejecutan de arriba a abajo
 - ▶ Cada vez que un usuario abre una pestaña del navegador que apunta a su aplicación, el script se vuelve a ejecutar
 - ▶ A medida que se ejecuta el script, Streamlit dibuja su salida en vivo en un navegador
 - ▶ Los scripts usan la memoria caché para evitar volver a calcular funciones costosas o cargar archivos grandes
 - ▶ Cada vez que un usuario interactúa con un widget, su secuencia de comandos se vuelve a ejecutar y el valor de salida de ese widget se establece en el nuevo valor durante esa ejecución
 - ▶ Las aplicaciones Streamlit pueden contener varias páginas, que se definen en archivos .py separados en una carpeta de páginas



- ▶ Streamlit facilita visualizar, cambiar y compartir datos.
- ▶ La referencia de la API está organizada por tipo de actividad
- ▶ Cada sección incluye métodos asociados con el tipo de actividad, incluidos ejemplos.
- ▶ <https://docs.streamlit.io/library/api-reference>



st.title

st.header

st.subheader

st.badge

st.caption

st.code

```
st.text
```

st.latex

st.divider

st.html

```
st.help
```

st.iframe

Widgets de Streamlit



► Visualización de datos:

`st.dataframe`

`st.data_editor`

`st.column_config`

`st.table`

`st.metric`

`st.json`

Dataframes

Display a dataframe as an interactive table.

```
st.dataframe(my_data_frame)
```

Static tables

Display a static table.

```
st.table(my_data_frame)
```

Data editor

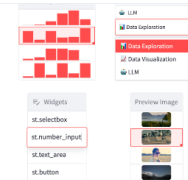
Display a data editor widget.

```
edited = st.data_editor(df, num_rows="")
```

Metrics

Display a metric in big bold font, with an optional indicator of how the metric changed.

```
st.metric("My metric", 42, 2)
```



Column configuration

Configure the display and editing behavior of dataframes and data editors.

```
st.column_config.NumberColumn("Price (
```

Dicts and JSON

Display object or string as a pretty-printed JSON string.

```
st.json(my_dict)
```



Widgets de Streamlit



► Gráficas:

`st.line_chart`

`st.area_chart`

`st.bar_chart`

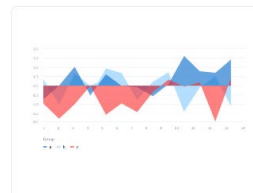
`st.pyplot (matplotlib)`

`st.map`

`st.scatter_chart`

`st.plotly_chart`
(interactivos)

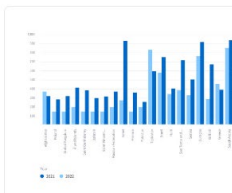
y otros: Altair, Vega,
Bokeh, PyDeck...



Simple area charts

Display an area chart.

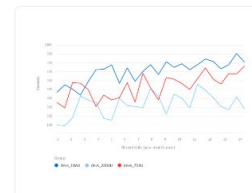
`st.area_chart(my_data_frame)`



Simple bar charts

Display a bar chart.

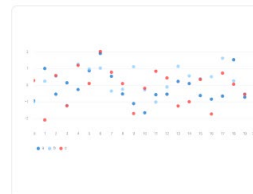
`st.bar_chart(my_data_frame)`



Simple line charts

Display a line chart.

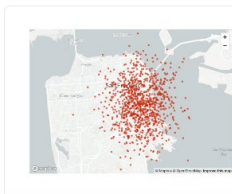
`st.line_chart(my_data_frame)`



Simple scatter charts

Display a line chart.

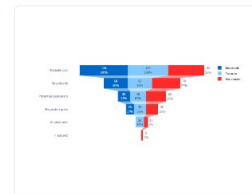
`st.scatter_chart(my_data_fræ)`



Scatterplots on maps

Display a map with points on it.

`st.map(my_data_frame)`



Plotly

Display an interactive Plotly chart.

`st.plotly_chart(my_plotly_cf)`



▶ Entrada (Botones):

`st.button`

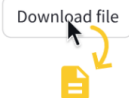
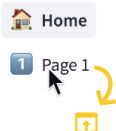
`st.download_button`

`st.form_submit_button`

`st.link_button`

`st.page_link`

`st.menu_button`

		
Button Display a button widget.	Download button Display a download button widget.	Form button Display a form submit button. For use with <code>st.form</code> .
<pre>clicked = st.button("Click me")</pre>	<pre>st.download_button("Download file", "file.pdf")</pre>	<pre>st.form_submit_button("Sign in")</pre>
		
Link button Display a link button.	Page link Display a link to another page in a multipage app.	Menu button Display a menu button.
<pre>st.link_button("Go to gallery", "https://www.google.com")</pre>	<pre>st.page_link("app.py", label="Home") st.page_link("pages/profile.py", label="Page 1")</pre>	<pre>st.menu_button("Export", options=["CSV", "JSON", "PDF"])</pre>



Widgets de Streamlit



► Entrada (Selección):

`st.checkbox`

`st.color_picker`

`st.multiselect`

`st.radio`

`st.selectbox`

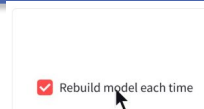
`st.select_slider`

`st.toggle`

`st.feedback`

`st.pills`

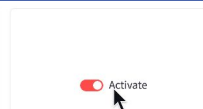
`st.segmented_control`



Checkbox

Display a checkbox widget.

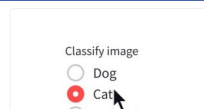
```
selected = st.checkbox("I agree")
```



Toggle

Display a toggle widget.

```
activated = st.toggle("Activate")
```



Radio

Display a radio button widget.

```
choice = st.radio("Pick one", ["Dog", "Cat", "Goldfish"])
```



Selectbox

Display a select widget.

```
choice = st.selectbox("Pick a color", ["red", "green", "blue"])
```



Multiselect

Display a multiselect widget. The multiselect widget starts as empty.

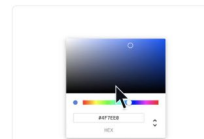
```
choices = st.multiselect("Pick some fruits", ["apples", "bananas", "peaches"])
```



Select slider

Display a slider widget to select items from a list.

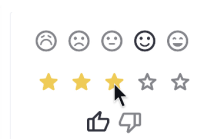
```
size = st.select_slider("Pick a size", ["small", "medium", "large"])
```



Color picker

Display a color picker widget.

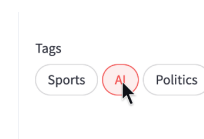
```
color = st.color_picker("Pick a color", "#FF69B4")
```



Feedback

Display a rating or sentiment button group.

```
st.feedback("stars")
```



Pills

Display a pill-button selection widget.

```
st.pills(["Tags"], ["Sports", "AI", "Politics"])
```



Segmented control

Display a segmented-button selection widget.

```
st.segmented_control("Filter", ["Open", "Closed", "All"])
```



Widgets de Streamlit



► Entrada (números y fechas):

`st.number_input`

`st.slider`

`st.date_input`

`st.time_input`

`st.datetime_input`

Schedule your event

2026/01/18:00

< January 2006 >

Su	Mo	Tu	We	Th	Fr	Sa
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

18:00

Datetime input
Display a datetime input widget.

```
datetime = st.datetime_input("Sch
```

Number of days

28 - +

Number input
Display a numeric input widget.

```
choice = st.number_input("Pi
```

Pick a number

0 42 100

Slider
Display a slider widget.

```
number = st.slider("Pick a r
```

Initial date

2019/07/06

July 2019

S	M	T	W	T	F	S
	1	2	3	4	5	
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Date input
Display a date input widget.

```
date = st.date_input("Your b
```

Set an alarm for

08:45

Time input
Display a time input widget.

```
time = st.time_input("Meetir
```



Widgets de Streamlit



► Entrada (Texto y Media):

`st.chat_input`

`st.text_area`

`st.text_input`

`st.audio_input`

`st.camera_input`

`st.data_editor`

`st.file_uploader`



Text input
Display a single-line text input widget.

```
name = st.text_input("First
```



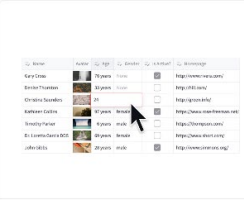
Text area
Display a multi-line text input widget.

```
text = st.text_area("Text to
```



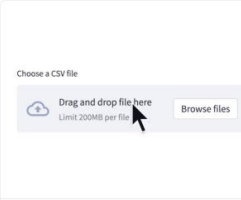
Chat input
Display a chat input widget.

```
prompt = st.chat_input("Say  
if prompt:  
    st.write(f"The user has
```



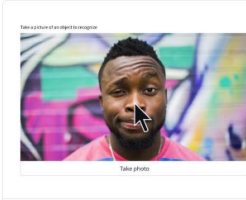
Data editor
Display a data editor widget.

```
edited = st.experimental_data
```



File uploader
Display a file uploader widget.

```
data = st.file_uploader("Uplo
```



Camera input
Display a widget that allows users to upload images directly from a camera.

```
image = st.camera_input("Take
```



Widgets de Streamlit



► Multimedia:

`st.image`

`st.audio`

`st.video`

`st.logo`

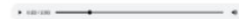
`st.pdf`



Image

Display an image or list of images.

```
st.image(numpy_array)
st.image(image_bytes)
st.image(file)
st.image("https://example.com/my_
```



Audio

Display an audio player.

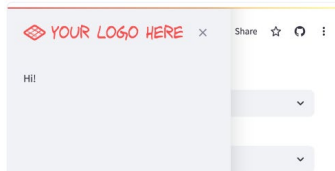
```
st.audio(numpy_array)
st.audio(audio_bytes)
st.audio(file)
st.audio("https://example.com/my_
```



Video

Display a video player.

```
st.video(numpy_array)
st.video(video_bytes)
st.video(file)
st.video("https://example.com/my_
```



Logo

Display a logo in the upper-left corner of your app and its sidebar.

```
st.logo("logo.jpg")
```



PDF

Display a PDF file.

```
st.pdf("my_document.pdf")
```



Widgets de Streamlit



► Diseño y contenedores:

`st.columns`

`st.container`

`st.dialog`

`st.empty`

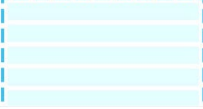
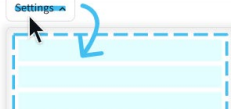
`st.expander`

`st.popover`

`st.sidebar`

`st.space`

`st.tabs`

 Columns Insert containers laid out as side-by-side columns. <pre>col1, col2 = st.columns(2) col1.write("this is column 1") col2.write("this is column 2")</pre>	 Container Insert a multi-element container. <pre>c = st.container() st.write("This will show last") c.write("This will show first") c.write("This will show second")</pre>	 Modal dialog Insert a modal dialog that can rerun independently from the rest of the script. <pre>@st.dialog("Sign up") def email_form(): name = st.text_input("Name") email = st.text_input("Email")</pre>
 Empty Insert a single-element container. <pre>c = st.empty() st.write("This will show last") c.write("This will be replaced") c.write("This will show first")</pre>	 Expander Insert a multi-element container that can be expanded/collapsed. <pre>with st.expander("Open to see more"): st.write("This is more content")</pre>	 Popover Insert a multi-element popover container that can be opened/closed. <pre>with st.popover("Settings"): st.checkbox("Show completed")</pre>
 Sidebar Display items in a sidebar. <pre>st.sidebar.write("This lives in the sidebar") st.sidebar.button("Click me!")</pre>	 Space Add vertical or horizontal space. <pre>st.space("small")</pre>	 Tabs Insert containers separated into tabs. <pre>tab1, tab2 = st.tabs(["Tab 1", "Tab2"]) tab1.write("this is tab 1") tab2.write("this is tab 2")</pre>



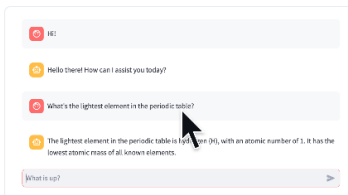
► Elementos de chat:

`st.chat_input`

`st.chat_message`

`st.status`

`st.write_stream`



Chat input
Display a chat input widget.

```
prompt = st.chat_input("Say something")
if prompt:
    st.write(f"The user has sent: {prompt}")
```

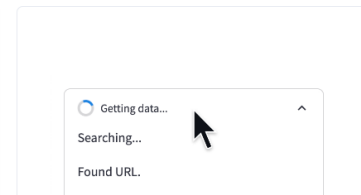
st.write_stream
Write generators or streams to the app with a typewriter effect.

```
st.write_stream(my_generator)
st.write_stream(my_llm_stream)
```



Chat message
Insert a chat message container.

```
import numpy as np
with st.chat_message("user"):
    st.write("Hello 🌟")
    st.line_chart(np.random.randn(30, 1))
```



Status container
Display output of long-running tasks in a container.

```
with st.status('Running'):
    do_something_slow()
```

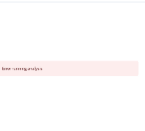
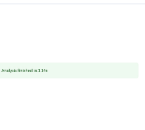


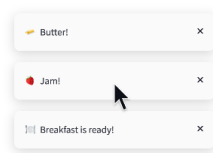
Widgets de Streamlit



► Progreso y estado:

`st.progress`
`st.spinner`
`st.toast`
`st.balloons`
`st.snow`
`st.error`
`st.warning`
`st.info`
`st.success`
`st.exception`
`st.status`

 Progress bar Display a progress bar. <pre>for i in range(101): st.progress(i) do_something_slow()</pre>	 Spinner Temporarily displays a message while executing a block of code. <pre>with st.spinner("Please wait..."): do_something_slow()</pre>	 Balloons Display celebratory balloons! <pre>do_something() # Celebrate when all done! st.balloons()</pre>
 Snowflakes Display celebratory snowflakes! <pre>do_something() # Celebrate when all done! st.snow()</pre>	 Error box Display error message. <pre>st.error("We encountered an error")</pre>	 Warning box Display warning message. <pre>st.warning("Unable to fetch image")</pre>
 Info box Display an informational message. <pre>st.info("Dataset is updated every 10 minutes")</pre>	 Success box Display a success message. <pre>st.success("Match found!")</pre>	 Exception output Display an exception. <pre>e = RuntimeError("This is an error") st.exception(e)</pre>



Toast
Briefly displays a toast message in the bottom-right corner.

```
st.toast('Butter!', icon='🧈')
```



► Control de flujo:

`st.fragment`

`st.stop`

`st.rerun`

`st.form`

`st.form_submit_button`

A veces es mejor para la experiencia del usuario esperar a que se complete un grupo de widgets relacionados antes de volver a ejecutar el script

Fragments

Define a fragment to rerun independently from the rest of the script.

```
@st.fragment(run_every="10s")
def fragment():
    df = get_data()
    st.line_chart(df)
```

Rerun script

Rerun the script immediately.

```
st.rerun()
```

Stop execution

Stops execution immediately.

```
st.stop()
```

Form submit button

Display a form submit button.

```
with st.form(key='my_form'):
    name = st.text_input("Name")
    email = st.text_input("Email")
    st.form_submit_button("Sign up")
```



► Marcadores, ayuda y opciones:

`st.set_page_config`

`st.echo`

`st.help`

`st.session_state`

`st.context`

`st.query_params`

Set page title, favicon, and more

Configures the default settings of the page.

```
st.set_page_config(  
    title="My app",  
    favicon=":shark:",  
)
```

Echo

Display some code on the app, then execute it. Useful for tutorials.

```
with st.echo():  
    st.write('This code will be pr
```

Get help

Display object's doc string, nicely formatted.

```
st.help(st.write)  
st.help(pd.DataFrame)
```

Context

`st.context` provides a read-only interface to access cookies, headers, locale, and other browser-session information.

```
st.context.cookies  
st.context.headers
```

Session State

Save data between reruns and across pages.

```
st.session_state["foo"] = "bar"
```

Query parameters

Get, set, or clear the query parameters that are shown in the browser's URL bar.

```
st.query_params[key] = value  
st.query_params.clear()
```



▶ Marcadores, ayuda y opciones:

`st.secrets`

`@st.cache_data`

- ▶ Se usa para almacenar en caché funciones que devuelven datos, para no tener que leerlos en cada ejecución

- ▶ Los objetos almacenados en caché pueden ser globales o estar limitados a la sesión. Los datos almacenados en caché de forma global están disponibles para todos los usuarios, sesiones y ejecuciones posteriores

`@st.cache_resources`

Secrets singleton

Access secrets from a local TOML file.

```
key = st.secrets["OpenAI_key"]
```

Secrets file

Save your secrets in a per-project or per-profile TOML file.

```
OpenAI_key = "<YOUR_SECRET_KEY>"
```

Cache data

Function decorator to cache functions that return data (e.g. dataframe transforms, database queries, ML inference).

```
@st.cache_data
def long_function(param1, param2):
    # Perform expensive computation here or
    # fetch data from the web here
    return data
```

Cache resource

Function decorator to cache functions that return global resources (e.g. database connections, ML models).

```
@st.cache_resource
def init_model():
    # Return a global resource here
    return pipeline(
        "sentiment-analysis",
        model="distilbert-base-uncased-finetuned-sst-2"
    )
```



- ▶ Entre los widgets comentados está:
`st.plotly_chart`
- ▶ Las gráficas que contiene son similares a Matplotlib, pero tienen la característica de ser interactivas, muy interesante para web
- ▶ En este enlace se puede ver todas las gráficas que se pueden utilizar:
 - ▶ <https://plotly.com/python/>
- ▶ Gráficas más sencillas de generar:
 - ▶ <https://plotly.com/python/plotly-express/>



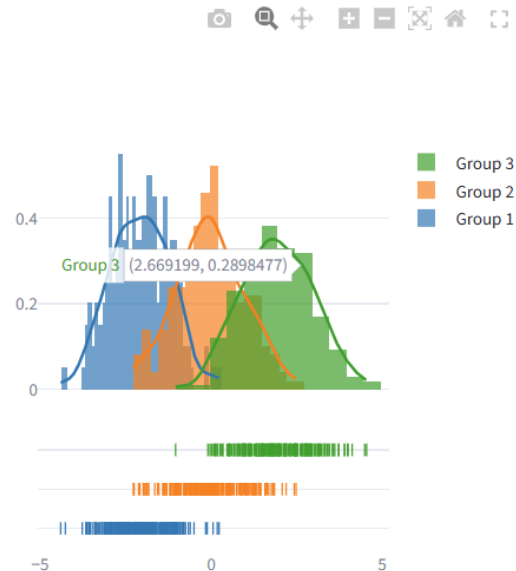
► Crear una gráfica de dispersión:

```
import plotly.figure_factory as ff
```

```
fig = ff.create_distplot(  
    hist_data, group_labels,  
    bin_size=[.1, .25, .5])
```

with izq:

```
st.plotly_chart(fig,  
    width='stretch',  
    theme="streamlit")
```



- ▶ La librería plotly express tiene datasets para testear:
 - ▶ <https://plotly.com/python-api-reference/generated/plotly.express.data.html>

```
import plotly.express as px
df = px.data.gapminder()
fig = px.scatter(df.query(f"year=={year}"), x="gdpPercap",
                 y="lifeExp", size="pop", color="continent",
                 hover_name="country", log_x=True, size_max=60)
st.plotly_chart(fig, theme="streamlit", width='stretch')
```

- ▶ Ejemplo de uso: Ejemplo2.py



- ▶ streamlit permite la visualización de mapas interactivos de Folium
- ▶ Instalar la librería:

```
pip install streamlit-folium
```
- ▶ Importar la librería:

```
from streamlit_folium import st_folium
```
- ▶ Crear el mapa Folium con todos sus componentes



- ▶ Incluirlo en streamlit:

```
st_data = st_folium(m, width=700, height=450)
```

- ▶ Devuelve un Diccionario con la información seleccionada
- ▶ Cada vez que el usuario interactúa con el mapa los valores de 'bounds', 'center' y 'zoom' se actualizan.
- ▶ Con estos valores podemos limitar los datos a mostrar en la parte visible, cambiar el tamaño de las marcas en función del zoom, etc.



Visualización de mapas en Streamlit



```
{"last_clicked":  
{"lat":39.52946653645165,"lng":-0.33302307128906256},  
"last_object_clicked":null,  
"last_object_clicked_tooltip":null,  
"last_object_clicked_popup":null,  
"all_drawings":null,  
"last_active_drawing":null,  
"bounds":{"_southWest":{"lat":39.28355960877959,"lng":-  
0.5142974853515626},"_northEast":{"lat":39.469330027492575,"lng":-  
0.2653884887695313}},  
"zoom":12,"last_circle_radius":null,"last_circle_polygon":null,  
"center":{"lat":39.376506610770406,"lng":-0.3898429870605469},  
"selected_layers":[],"selected_tags":[]}
```

Ejemplo3.py



- ▶ SJE es un componente Streamlit personalizado, creado para evaluar expresiones arbitrarias de Javascript y devolver el resultado.
- ▶ Realiza funcionalidades que son simples en JavaScript, pero no disponibles o difíciles de hacer en Streamlit.
- ▶ Ejemplos: Obtener el ancho del dispositivo, obtener el idioma del navegador, verificar el estado de la batería y la red, obtener la ubicación en un mapa...
 - ▶ <https://discuss.streamlit.io/t/new-component-streamlit-js-eval/30861>
 - ▶ <https://pypi.org/project/streamlit-js-eval/>



Ejemplo paso a paso



- Crear un mapa de coropletas de las provincias españolas en el que se muestre el % de paro para hombres, mujeres y ambos sexos para cada trimestre, y una gráfica de barras con los últimos 12 trimestres

Sexo

☒ Ambos sexos

☐ Hombres

☐ Mujeres

Año

2025

Trimestre

☐ 1

☐ 2

☐ 3

☒ 4

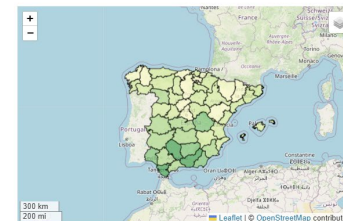
Provincia

Total Nacional

Paro por provincias, sexo y trimestre

Fuente: Ine

2025 T4 - Ambos sexos



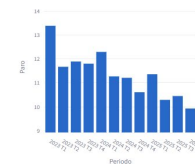
Datos de paro: Total Nacional

Ambos sexos
9.93 %

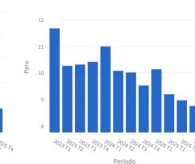
Hombres
8.76 %

Mujeres
11.24 %

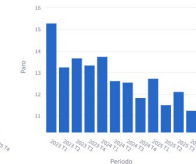
Total Nacional - Ambos sexos



Total Nacional - Hombres



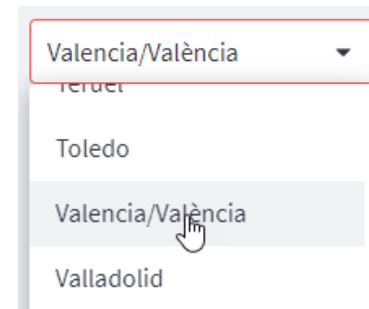
Total Nacional - Mujeres



Ejemplo paso a paso



- ▶ La provincia a mostrar se puede seleccionar desde el menú desplegable o pinchando en el mapa
- ▶ El nombre de la provincia aparecerá al pasar el ratón sobre ella



Datos de paro: Valencia/València

Ambos sexos	Hombres	Mujeres
12.05 %	11.59 %	12.56 %



Ejemplo paso a paso



- ▶ En la barra lateral se podrá elegir el sexo a mostrar
- ▶ También se selecciona el año y el trimestre
- ▶ Tened en cuenta que es posible que el último año no tenga todos los trimestres

Sexo

☒ Ambos sexos

☐ Hombres

☐ Mujeres

2021 ▼

Trimestre

☒ 1

☐ 2

☐ 3

☐ 4



Ejemplo paso a paso



- ▶ Se proporciona el fichero `provincias.json` con los polígonos de cada provincia
- ▶ Cada provincia tiene un código: `properties.código`
- ▶ También se proporciona el fichero `TasaParo.csv` que contiene la tasa de paro por año, trimestre, provincia (incluido Total Nacional) y Sexo (incluido Ambos sexos)
- ▶ Los datos van desde el T1 de 2002 al T4 de 2025



- ▶ A tener en cuenta:
 - ▶ En el fichero json cada provincia tiene un código asignado, que coincide con el código del fichero csv
 - ▶ El código es de tipo str con cero a la izquierda
 - ▶ A Total Nacional se le asigna el código 00 (valor por defecto)
 - ▶ En el csv el código es entero
- ▶ Datos:
 - ▶ <https://www.ine.es/jaxiT3/Tabla.htm?t=65349>



► Cargar datos en caché:

@st.cache_data

```
def cargar_datos():
```

```
    prov_geo = path+'provincias.geojson'
```

```
    fich_paro = path+'TasaParo.csv'
```

```
    prov_data = pd.read_csv(fich_paro, encoding='latin1')
```

```
    #Sexo          Codigo    Provincia Trimestre Paro
```

```
#El código de provincia en el geojson es str y con cero a la izquierda
```

```
    prov_data['codigo'] = prov_data['codigo'].astype(str).str.zfill(2)
```

```
    prov_list = list(prov_data['Provincia'].unique())
```

```
    prov_dict = pd.Series(prov_data.Provincia.values,  
                          index=prov_data.codigo).to_dict()
```

```
    return prov_geo, prov_data, prov_list, prov_dict
```



Ejemplo paso a paso



► Crear barra lateral

```
def display_time_filters(df, sex):  
    year_list = list(df['Año'].unique())  
    year_list.sort(reverse = True )  
    year = st.sidebar.selectbox('Año', year_list, 0)  
    quarter = st.sidebar.radio('Trimestre', [1, 2, 3, 4], 3)  
    st.header(f'{year} T{quarter} - {sex}' )  
    return year, quarter
```

```
def display_prov_filter(df, prov):  
    return st.sidebar.selectbox('Provincia', prov_list)
```

```
def display_sex_filter():  
    return st.sidebar.radio('Sexo', ['Ambos sexos', 'Hombres', 'Mujeres'])
```

The screenshot shows a sidebar filter interface with three sections:

- Sexo:** Three radio buttons. 'Ambos sexos' is selected (indicated by a red dot). The other options are 'Hombres' and 'Mujeres'.
- Año:** A dropdown menu showing '2025'.
- Trimestre:** Four radio buttons. '4' is selected (indicated by a red dot). The other options are '1', '2', and '3'.
- Provincia:** A dropdown menu showing 'Total Nacional'.

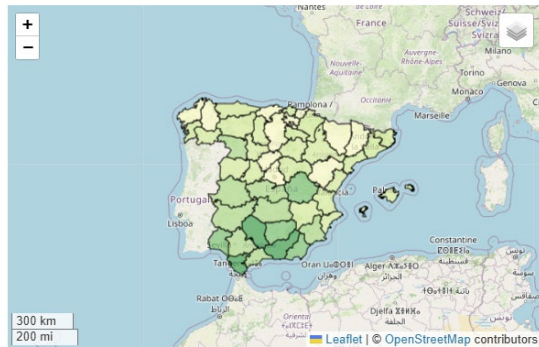


Ejemplo paso a paso



► Dibujar el mapa:

```
def display_map(df, year, quarter, sex):  
    df = df[(df['Año'] == year) & (df['Trimestre'] == quarter) & (df['Sexo'] == sex)]  
  
    m = folium.Map(location=[40.42, -3.7], zoom_start=5, control_scale=True)  
    coropletas = folium.Choropleth(geo_data=prov_geo,name="choropleth",data=df,  
                                   columns=("codigo", "Paro"),key_on="properties.codigo", fill_color='YlGn',  
                                   fill_opacity=0.7,line_opacity=1.0,legend_name="Tasa de Paro (%)")  
    coropletas.geojson.add_to(m)  
    for feature in coropletas.geojson.data['features']:  
        code = feature['properties']['codigo']  
        feature['properties']['Provincia'] = prov_dict[code]  
    coropletas.geojson.add_child(folium.features.GeoJsonTooltip(['Provincia'], labels=False))  
  
    folium.LayerControl().add_to(m)  
    st_map = st_folium(m, width=700, height=450)  
    codigo = '00'  
    if st_map['last_active_drawing']:  
        codigo = st_map['last_active_drawing']['properties']['codigo']  
    return codigo
```



► Visualizar los datos:

```
def display_datos_paro(df, year, quarter, sex, prov_name):  
    df = df[(df['Año'] == year) & (df['Trimestre'] == quarter) &  
            (df['Sexo'] == sex) & (df['Provincia'] == prov_name)]  
    st.metric(sex, str(df.Paro.iat[0]))+' %')
```

Datos de paro: Total Nacional

Ambos sexos

9.93 %

Hombres

8.76 %

Mujeres

11.24 %



Ejemplo paso a paso



- Dibujar gráficas:

```
def grafica_paro(df, year, quarter, prov_name, prov_code, sex):  
    df = df[(df['codigo'] == prov_code) &  
            (df['Sexo'] == sex)].reset_index()  
    idx = (df.index[(df['Año'] == year) &  
                   (df['Trimestre'] == quarter)].tolist())[0]  
    df = df.iloc[idx:min(len(df), idx+12)]  
    df = df.sort_values(by=['Año', 'Trimestre'], ascending=True)  
    df['Periodo'] = df['Año'].astype(str) + ' T' +  
                   df['Trimestre'].astype(str)  
    if '/' in prov_name:  
        prov_name = prov_name.split('/')[0]  
    fig = px.bar(df, x='Periodo', y='Paro', title=f'{prov_name} - {sex}')  
    fig.update_layout(yaxis=dict(range=[df['Paro'].min() - 1,  
                                         df['Paro'].max() + 1]))  
    st.plotly_chart(fig)
```



Ejemplo paso a paso



► Programa principal

```
st.set_page_config(APP_TITLE,layout="wide")  
st.title(APP_TITLE)  
st.caption(APP_SUB_TITLE)
```

```
prov_geo, prov_data, prov_list, prov_dict = cargar_datos()
```

```
sex = display_sex_filter()  
year, quarter = display_time_filters(prov_data, sex)  
prov_code = display_map(prov_data, year, quarter, sex)  
prov_name = display_prov_filter(prov_data, '')
```



► Programa principal

```
if (prov_code!='00'):
    prov_name = prov_dict[prov_code]
    st.subheader(f'Datos de paro: {prov_name}')
    col1, col2, col3 = st.columns(3)
    with col1:
        display_datos_paro(prov_data, year, quarter, 'Ambos sexos', prov_name)
        grafica_paro(prov_data, year, quarter, prov_name, prov_code, 'Ambos sexos')
    with col2:
        display_datos_paro(prov_data, year, quarter, 'Hombres', prov_name)
        grafica_paro(prov_data, year, quarter, prov_name, prov_code, 'Hombres')
    with col3:
        display_datos_paro(prov_data, year, quarter, 'Mujeres', prov_name)
        grafica_paro(prov_data, year, quarter, prov_name, prov_code, 'Mujeres')
```

Ejemplo4.py

